

Trilha 8 - Python Backend + IA

Plano técnico completo para evolução até nível sênior

Documento técnico formatado com conteúdo integral da trilha.

Python Backend | IA aplicada | MLOps | RAG | Arquitetura | Produção

Objetivo do documento

Organizar a trilha em formato profissional para estudo, revisão, execução prática e evolução técnica até nível sênior.

Sumário

Lista das principais seções e subseções do documento, com numeração de páginas.

1. Análise inicial do conteúdo	6
Tema central	6
Objetivo principal da trilha	6
Competências técnicas a desenvolver	6
Classificação por nível	6
2. Explicação técnica dos grandes tópicos	7
2.1 Python Core avançado	7
Principais tópicos:	7
2.2 FastAPI	9
Tópicos principais:	9
2.3 Django e Django REST Framework avançado	11
Tópicos principais:	11
2.4 Async Python avançado	14
Tópicos principais:	14
2.5 Performance Python	15
Tópicos principais:	15
2.6 Dados e pipelines	17
Tópicos principais:	17
2.7 ML, PyTorch e MLOps	19
Tópicos principais:	19
2.8 LLMs e RAG com Python	21
Tópicos principais:	22
2.9 Testes, segurança e observabilidade	24
Tópicos principais:	24
3. Organização como plano de estudos	26
Módulo 1 - Fundamentos sólidos de Python	26
Objetivo do módulo	26
Conteúdos a estudar	26
Ordem recomendada	27
Conceitos obrigatórios	27
Práticas recomendadas	27
Exercícios técnicos	27
Mini projeto	27
Funcionalidades:	27
Critérios de conclusão	28
Nível esperado ao finalizar	28
Módulo 2 - Aplicação prática com FastAPI e Django/DRF	28

Objetivo do módulo	28
Conteúdos a estudar	28
FastAPI:	28
Django/DRF:	28
Ordem recomendada	28
Conceitos obrigatórios	29
Práticas recomendadas	29
Exercícios técnicos	29
Mini projeto	29
Funcionalidades:	29
Critérios de conclusão	30
Nível esperado ao finalizar	30
Módulo 3 - Aprofundamento em async, performance e dados	30
Objetivo do módulo	30
Conteúdos a estudar	30
Async:	30
Performance:	30
Dados:	30
Ordem recomendada	31
Conceitos obrigatórios	31
Práticas recomendadas	31
Exercícios técnicos	31
Mini projeto	32
Funcionalidades:	32
Critérios de conclusão	32
Nível esperado ao finalizar	32
Módulo 4 - Arquitetura, boas práticas e integração	32
Objetivo do módulo	32
Conteúdos a estudar	32
Ordem recomendada	33
Conceitos obrigatórios	33
Práticas recomendadas	33
Exercícios técnicos	33
Mini projeto	33
Funcionalidades:	33
Critérios de conclusão	34
Nível esperado ao finalizar	34
Módulo 5 - Produção, performance, segurança e escalabilidade	34
Objetivo do módulo	34
Conteúdos a estudar	34
Ordem recomendada	34

Conceitos obrigatórios	35
Práticas recomendadas	35
Exercícios técnicos	35
Mini projeto	35
Funcionalidades:	35
Critérios de conclusão	36
Nível esperado ao finalizar	36
Módulo 6 - Domínio sênior, IA aplicada e liderança técnica	36
Objetivo do módulo	36
Conteúdos a estudar	36
ML/MLOps:	36
LLM/RAG:	36
Liderança técnica:	37
Ordem recomendada	37
Conceitos obrigatórios	37
Práticas recomendadas	38
Exercícios técnicos	38
Mini projeto	38
Funcionalidades:	38
Critérios de conclusão	38
Nível esperado ao finalizar	39
4. Critérios de domínio técnico por grande área	39
Python Core	39
Backend Python	39
Async e performance	39
Dados e pipelines	39
ML/MLOps	39
LLM/RAG	40
Segurança e observabilidade	40
5. Aplicação prática em projetos reais	40
APIs	40
Banco de dados	40
Microserviços	41
Segurança	41
Performance	42
Manutenção evolutiva	43
6. Profundidade sênior: trade-offs e decisões	43
FastAPI vs Django/DRF	43
Async vs fila	43
Pandas vs Polars	43

RAG simples vs RAG avançado	44
Modelo próprio vs API externa	44
7. Cronograma sugerido	44
Plano intensivo de 24 semanas	44
Semanas 1-4 - Python Core avançado	44
Semanas 5-8 - FastAPI	45
Semanas 9-11 - Django/DRF	45
Semanas 12-14 - Async e performance	45
Semanas 15-17 - Dados e pipelines	46
Semanas 18-20 - ML e MLOps	46
Semanas 21-23 - LLMs e RAG	46
Semana 24 - Projeto final e revisão sênior	47
8. Projetos recomendados para portfólio	47
Projeto 1 - API Python profissional com FastAPI	47
Projeto 2 - Pipeline financeiro com validação	47
Projeto 3 - Serviço de ML com MLOps	48
Projeto 4 - Plataforma RAG corporativa	48
Projeto 5 - Microserviço Python integrado com Java/Spring	48
9. Checklist de domínio técnico	49
Python Core	49
Dados	49
ML/MLOps	50
LLM/RAG	50
10. Indicadores de prontidão para nível	50

1. Análise inicial do conteúdo

Tema central

A trilha é focada em Python aplicado a backend moderno, automação, engenharia de dados, IA aplicada, LLMs, RAG, MLOps, segurança, observabilidade e integração com ecossistemas Java/cloud.

Ela não trata Python apenas como linguagem de script. O objetivo é formar um profissional capaz de usar Python como linguagem principal para sistemas backend, APIs, pipelines de dados, serviços de IA, produtos com LLMs e aplicações observáveis em produção.

Objetivo principal da trilha

Formar domínio técnico para construir, manter e evoluir sistemas Python em ambientes reais, incluindo:

- APIs REST e assíncronas.
- Serviços backend com FastAPI e Django/DRF.
- Integração com bancos SQL/NoSQL.
- Pipelines de dados.
- Aplicações com Machine Learning.
- APIs com LLM, embeddings e RAG.
- Observabilidade, segurança, testes e performance.
- Deploy em cloud.
- Integração com sistemas Java/Spring, mensageria e microserviços.

Competências técnicas a desenvolver

Ao longo da trilha, o estudante deve desenvolver competências em:

Área	Competência esperada
Python Core	Escrever código Python limpo, tipado, modular, testável e performático
Backend	Projetar APIs com FastAPI, Django e DRF
Banco de dados	Modelar, consultar, migrar e otimizar dados
Async	Usar programação assíncrona com critério técnico
Performance	Medir gargalos e otimizar CPU, memória e I/O
Dados	Criar pipelines robustos, validados e rastreáveis
IA/ML	Criar, treinar, versionar e servir modelos
LLM/RAG	Construir sistemas com embeddings, vector stores, agentes e avaliação
Segurança	Mitigar vulnerabilidades comuns em aplicações Python
Observabilidade	Implementar logs, métricas, tracing e alertas
Arquitetura	Tomar decisões técnicas justificáveis
Liderança técnica	Revisar código, definir padrões e orientar evolução de sistemas

Classificação por nível

Nível	Conteúdos principais
Básico	Python Core, ambientes virtuais, OOP, testes básicos, SQL inicial
Intermediário	FastAPI, Django/DRF, SQLAlchemy, Alembic, pytest, Pandas, async básico
Avançado	Performance, async avançado, pipelines, MLflow, PyTorch, RAG, segurança
Sênior	Arquitetura de serviços, MLOps, observabilidade, trade-offs, escalabilidade, governança técnica, integração cloud/java

2. Explicação técnica dos grandes tópicos

2.1 Python Core avançado

O que é

Python Core avançado é o domínio profundo da linguagem Python além da sintaxe básica. Inclui tipagem, orientação a objetos, protocolos, generics, iteradores, geradores, decorators, context managers, async/await, empacotamento, qualidade de código e testes.

Principais tópicos:

- typing
- Protocol
- Generic
- Literal
- TypedDict
- dataclasses
- Pydantic
- OOP
- Magic methods
- Iterators
- Generators
- Decorators
- Context managers
- Async/await
- Packaging
- Virtual environments
- Poetry
- uv
- Ruff
- mypy
- pyright
- pytest

Para que serve

Serve para construir código Python com maior previsibilidade, legibilidade e segurança técnica. Em sistemas reais, Python mal estruturado rapidamente se torna difícil de manter, testar e evoluir.

O Python Core avançado permite:

- Criar APIs organizadas.
- Reduzir bugs por erro de tipo.

- Padronizar contratos internos.
- Criar bibliotecas reutilizáveis.
- Melhorar testes.
- Evitar acoplamento excessivo.
- Organizar projetos escaláveis.

Por que é importante

Python é uma linguagem dinâmica. Isso traz produtividade, mas também aumenta riscos em projetos grandes.

Sem tipagem, organização e testes, erros aparecem tarde, geralmente em produção.

Um profissional sênior precisa compensar a flexibilidade da linguagem com:

- Tipagem estática.
- Testes automatizados.
- Contratos claros.
- Arquitetura modular.
- Observabilidade.
- Revisões de código consistentes.

Onde é usado em projetos reais

Python Core avançado aparece em praticamente qualquer projeto sério:

- APIs FastAPI.
- Serviços Django.
- SDKs internos.
- Workers assíncronos.
- Pipelines de dados.
- Integrações com Kafka.
- Serviços de IA.
- Ferramentas CLI.
- Libs compartilhadas entre times.
- Automação de infraestrutura.

Quais problemas resolve

Resolve problemas como:

- Código difícil de testar.
- Erros de tipo em runtime.
- Duplicidade de lógica.
- Baixa manutenibilidade.
- Baixa previsibilidade.
- Interfaces implícitas demais.
- Alto custo de evolução.
- Falta de padronização entre projetos.

Erros comuns

Evite:

- Usar Python como "script solto" em sistemas grandes.
- Ignorar tipagem.
- Misturar regra de negócio com framework.
- Criar funções enormes.
- Usar dict genérico para tudo.
- Abusar de herança.

- Não configurar pytest.
- Não usar lint/formatter.
- Não separar domínio, infraestrutura e aplicação.
- Criar decorators mágicos difíceis de debugar.

Conhecimentos prévios necessários

Antes de avançar, é necessário dominar:

- Sintaxe Python.
- Funções.
- Classes.
- Módulos.
- Exceções.
- List/dict/set/tuple.
- Compreensão de listas.
- Ambiente virtual.
- Instalação de pacotes.
- Testes básicos.

Profundidade esperada para nível sênior

Um profissional sênior deve ser capaz de:

- Projetar pacotes Python reutilizáveis.
- Definir padrões de tipagem para o time.
- Revisar código com foco em manutenibilidade.
- Decidir quando usar dataclass, Pydantic, TypedDict ou classe comum.
- Aplicar Protocol para reduzir acoplamento.
- Usar Generic quando houver reuso real de abstrações.
- Configurar mypy, pyright, ruff, pytest e CI.
- Evitar abstrações excessivas.
- Justificar decisões de design.

2.2 FastAPI

O que é

FastAPI é um framework moderno para construção de APIs Python, com foco em performance, tipagem, documentação automática e suporte nativo a async.

Tópicos principais:

- Routing.
- Dependency Injection.
- Pydantic v2.
- Middleware.
- Exception handlers.
- OpenAPI.
- Auth.
- JWT.
- OAuth2.
- Background tasks.
- Async endpoints.
- SQLAlchemy.
- Alembic.

- TestClient.
- Observabilidade.
- Rate limiting.
- CORS.
- Uploads.

Para que serve

Serve para construir APIs HTTP modernas com alta produtividade e boa performance. É muito usado para:

- APIs REST.
- APIs de IA.
- Microsserviços.
- Gateways internos.
- Serviços assíncronos.
- APIs de ingestão de dados.
- Serviços de busca semântica.
- Produtos baseados em LLM.

Por que é importante

FastAPI combina três pontos fortes:

- Produtividade: criação rápida de endpoints.
- Contratos claros: schemas com Pydantic e OpenAPI.
- Performance: suporte a async e ASGI.

Para aplicações de IA, FastAPI é muito útil porque permite expor modelos, pipelines, RAG e automações por meio de APIs bem documentadas.

Onde é usado em projetos reais

Exemplos:

- API para upload de documentos.
- API de embeddings.
- API de busca semântica.
- API de chatbot corporativo.
- API para classificação automática.
- API para integração com serviços Java.
- Microsserviço de recomendação.
- Serviço de scoring antifraude.
- Backend para dashboards de IA.

Quais problemas resolve

Resolve:

- Exposição de modelos de IA via HTTP.
- Padronização de contratos.
- Documentação automática.
- Validação de entrada.
- Separação entre rota, serviço e schema.
- Integração com autenticação.
- Testes de endpoints.
- Execução de tarefas em background.

Erros comuns

Evite:

- Colocar regra de negócio diretamente no endpoint.
- Usar async sem entender o impacto.
- Bloquear o event loop com operações síncronas pesadas.
- Não versionar API.
- Não configurar CORS corretamente.
- Não tratar exceções globalmente.
- Não limitar tamanho de upload.
- Não proteger endpoints sensíveis.
- Não implementar logs estruturados.
- Não usar migrations com Alembic.

Conhecimentos prévios necessários

Python Core.

HTTP.

REST.

JSON.

Tipagem.

Pydantic.

Banco de dados.

SQL básico.

Testes com pytest.

Conceitos de autenticação.

Profundidade esperada para nível sênior

Um sênior precisa saber:

- Projetar APIs com separação clara entre routers, services, repositories e schemas.
- Definir contratos OpenAPI consistentes.
- Implementar autenticação JWT/OAuth2.
- Aplicar rate limiting.
- Tratar upload com segurança.
- Integrar com banco, fila, cache e storage.
- Adicionar logs, métricas e tracing.
- Testar endpoints com fixtures e mocks.
- Decidir entre endpoint síncrono e assíncrono.
- Preparar a API para produção com Uvicorn/Gunicorn, health checks e observabilidade.

2.3 Django e Django REST Framework avançado

O que é

Django é um framework web completo. Django REST Framework, ou DRF, é uma extensão para criação de APIs REST usando Django.

Tópicos principais:

- Models.
- Migrations.
- Admin.
- QuerySets.
- Managers.

- Signals.
- Serializers.
- ViewSets.
- Permissions.
- Throttling.
- Filtering.
- Pagination.
- select_related.
- prefetch_related.
- Multi-database.
- Transactions.
- Celery.
- Caching.
- Security.

Para que serve

Django serve para construir aplicações web e APIs com muitos recursos prontos:

- ORM.
- Autenticação.
- Admin.
- Migrations.
- Segurança padrão.
- Forms.
- Middleware.
- Sessões.
- Permissões.

DRF serve para expor esses dados via APIs robustas.

Por que é importante

Django é uma excelente escolha quando o projeto precisa de:

- Backoffice.
- Admin rápido.
- CRUD robusto.
- Regras de permissão.
- Produtividade.
- ORM maduro.
- Segurança pronta.
- Aplicações monolíticas bem estruturadas.

Onde é usado em projetos reais

Exemplos:

- Sistemas administrativos.
- ERPs internos.
- Plataformas SaaS.
- Sistemas educacionais.
- Marketplaces.
- Sistemas financeiros.
- Backoffice para operações.
- APIs CRUD corporativas.

- Plataformas com autenticação e permissões complexas.

Quais problemas resolve

Resolve:

- Criação rápida de aplicações completas.
- Administração de dados.
- Mapeamento objeto-relacional.
- Controle de autenticação.
- Gestão de permissões.
- Paginação, filtros e serializers.
- Operações transacionais.
- Segurança básica contra ataques comuns.

Erros comuns

Evite:

- Criar lógica pesada dentro de models ou views sem critério.
- Ignorar N+1 queries.
- Usar signals para regra de negócio crítica sem rastreabilidade.
- Criar serializers gigantes.
- Não usar transações.
- Não configurar cache.
- Não revisar permissões.
- Usar Django Admin como produto final sem governança.
- Fazer queries ineficientes.
- Ignorar índices no banco.

Conhecimentos prévios necessários

Python OOP.

SQL.

HTTP.

MVC/MTV.

REST.

Banco relacional.

Migrations.

Testes.

Autenticação.

Profundidade esperada para nível sênior

Um sênior precisa:

- Otimizar QuerySets.
- Identificar e resolver N+1 com `select_related` e `prefetch_related`.
- Usar transactions corretamente.
- Definir boundaries entre app, service layer e domain layer.
- Projetar permissões granulares.
- Aplicar caching com critério.
- Evitar acoplamento excessivo ao ORM.
- Decidir entre Django monolith, modular monolith ou microserviços.
- Garantir segurança, observabilidade e performance.

2.4 Async Python avançado

O que é

Async Python é o uso de programação assíncrona para lidar melhor com operações de I/O, como chamadas HTTP, banco assíncrono, filas, sockets e streams.

Tópicos principais:

- Event loop.
- Coroutines.
- Tasks.
- Futures.
- Async context managers.
- Async generators.
- Cancellation.
- Timeouts.
- Backpressure.
- anyio.
- trio.
- aiohttp.
- httpx async.
- async SQLAlchemy.

Para que serve

Serve para melhorar throughput em aplicações que fazem muitas operações de entrada e saída, como:

- Chamadas para APIs externas.
- Consumo de serviços.
- Upload/download.
- WebSockets.
- Processamento concorrente de requisições.
- Integração com LLMs.
- Orquestração de múltiplos serviços.

Por que é importante

Sistemas modernos dependem de muitas integrações. Uma API pode consultar banco, cache, fila, storage, serviço externo e LLM na mesma requisição.

Async ajuda quando o gargalo é espera, não CPU.

Onde é usado em projetos reais

Exemplos:

- FastAPI com endpoints async.
- Chamadas paralelas para APIs externas.
- Integração com OpenAI, Anthropic, Gemini ou modelos locais.
- Busca em vector stores.
- Workers de ingestão.
- Crawlers.
- WebSockets.
- Serviços de streaming.
- Gateways de integração.

Quais problemas resolve

Resolve:

- Baixo throughput por espera de I/O.
- Lentidão em chamadas externas.
- Uso ineficiente de threads.
- Processamento concorrente de tarefas leves.
- Orquestração de chamadas paralelas.

Erros comuns

Evite:

- Usar async para CPU-bound.
- Chamar função bloqueante dentro de endpoint async.
- Não configurar timeouts.
- Não tratar cancellation.
- Criar tasks sem controle.
- Não aplicar backpressure.
- Misturar libs sync e async sem cuidado.
- Achar que async sempre melhora performance.
- Não limitar concorrência.

Conhecimentos prévios necessários

Python intermediário.

HTTP.

Threads/processos.

I/O.

FastAPI.

Banco de dados.

Testes.

Noções de concorrência.

Profundidade esperada para nível sênior

Um sênior precisa:

- Decidir quando usar async, threading, multiprocessing ou fila.
- Evitar bloquear event loop.
- Configurar timeouts e retries.
- Projetar controle de concorrência.
- Aplicar backpressure.
- Isolar operações CPU-bound.
- Diagnosticar deadlocks, starvation e vazamentos de tasks.
- Avaliar impacto em produção.

2.5 Performance Python

O que é

Performance Python é a disciplina de medir, diagnosticar e otimizar aplicações Python em CPU, memória, I/O, concorrência e execução de modelos.

Tópicos principais:

- cProfile.
- Py-Spy.
- Scalene.

- Memory profiling.
- Vectorização.
- NumPy.
- Polars.
- Cython.
- Numba.
- Multiprocessing.
- Threading.
- GIL.
- C extensions.
- ONNX Runtime.

Para que serve

Serve para identificar gargalos reais e otimizar sistemas Python sem achismo.

É usada para:

- Reduzir latência.
- Aumentar throughput.
- Diminuir consumo de memória.
- Reduzir custo de infraestrutura.
- Melhorar processamento de dados.
- Servir modelos com menor latência.

Por que é importante

Python é produtivo, mas pode ser lento se usado de forma ingênua. Em produção, performance afeta:

- Custo cloud.
- Experiência do usuário.
- SLA.
- Escalabilidade.
- Estabilidade.
- Capacidade de processamento.

Onde é usado em projetos reais

Exemplos:

- Pipeline financeiro com milhões de linhas.
- API de IA com resposta em baixa latência.
- Pré-processamento de dados.
- Serviço de recomendação.
- Batch jobs.
- Inferência com modelos ONNX.
- Processamento paralelo.
- Otimização de DataFrames.

Quais problemas resolve

Resolve:

- Lentidão de loops Python.
- Alto consumo de memória.
- Processamento batch demorado.
- APIs com latência alta.
- Uso ineficiente de CPU.
- Gargalos em serialização.

- Sobrecarga por chamadas externas.

Erros comuns

Evite:

- Otimizar sem medir.
- Usar Pandas para tudo.
- Ignorar Polars ou NumPy.
- Usar threading para CPU-bound esperando ganho.
- Ignorar o GIL.
- Carregar datasets inteiros na memória sem necessidade.
- Não usar streaming/chunking.
- Não avaliar custo cloud.
- Não medir p95/p99 de latência.

Conhecimentos prévios necessários

Python intermediário.

Estruturas de dados.

Sistemas operacionais.

CPU vs I/O.

Pandas/NumPy.

Testes.

Métricas de performance.

Profundidade esperada para nível sênior

Um sênior precisa:

- Medir antes de otimizar.
- Separar gargalo de CPU, memória, I/O e rede.
- Escolher entre NumPy, Polars, Cython, Numba, multiprocessing ou reescrita parcial.
- Avaliar ONNX Runtime para inferência.
- Definir SLIs/SLOs de performance.
- Justificar custo-benefício da otimização.
- Evitar otimização prematura.

2.6 Dados e pipelines

O que é

Dados e pipelines envolvem ingestão, validação, transformação, enriquecimento, armazenamento e disponibilização de dados para analytics, IA, relatórios ou sistemas operacionais.

Tópicos principais:

- NumPy.
- Pandas.
- Polars.
- SQL.
- EDA.
- Feature engineering.
- Airflow.
- Prefect.
- Dagster.
- dbt.

- Data validation.
- Great Expectations.
- Batch processing.
- Streaming básico.
- Data quality.

Para que serve

Serve para transformar dados brutos em dados confiáveis, utilizáveis e auditáveis.

Pipelines são usados para:

- Importar arquivos.
- Normalizar dados.
- Validar qualidade.
- Popular bancos analíticos.
- Preparar features para ML.
- Gerar relatórios.
- Alimentar dashboards.
- Integrar sistemas.

Por que é importante

IA depende de dados. Sem dados confiáveis, o modelo falha. Em projetos reais, muitos problemas de IA são, na verdade, problemas de engenharia de dados.

Um pipeline ruim gera:

- Dados duplicados.
- Dados inconsistentes.
- Falhas silenciosas.
- Métricas erradas.
- Modelos enviesados.
- Decisões incorretas.

Onde é usado em projetos reais**Exemplos:**

- Pipeline de ingestão financeira.
- ETL de transações.
- Importação CSV/OFX.
- Enriquecimento de dados de clientes.
- Feature store.
- Pipeline de treinamento ML.
- Dados para dashboards.
- Integração com data lake.
- Auditoria de eventos.

Quais problemas resolve**Resolve:**

- Dados inconsistentes.
- Falta de rastreabilidade.
- Transformações manuais.
- Falta de validação.
- Processamento repetitivo.
- Falha em carga incremental.
- Baixa confiabilidade em dados usados por IA.

Erros comuns

Evite:

- Não validar schema.
- Não tratar dados ausentes.
- Não versionar transformação.
- Não registrar lineage.
- Não testar pipeline.
- Não controlar idempotência.
- Não monitorar falhas.
- Misturar regra de negócio com transformação ad hoc.
- Reprocessar tudo sem necessidade.

Conhecimentos prévios necessários

SQL.

Python.

Pandas/Polars.

Banco de dados.

Modelagem de dados.

Conceitos de ETL/ELT.

Testes.

Versionamento.

Profundidade esperada para nível sênior

Um sênior precisa:

- Projetar pipelines idempotentes.
- Definir validações de qualidade.
- Separar bronze/silver/gold quando aplicável.
- Escolher entre batch e streaming.
- Definir orquestração com Airflow, Prefect ou Dagster.
- Avaliar dbt para transformação SQL.
- Garantir rastreabilidade e reprocessamento seguro.
- Controlar custo computacional.
- Conectar dados com objetivos de produto e IA.

2.7 ML, PyTorch e MLOps

O que é

Essa área cobre criação, treinamento, validação, versionamento, deploy e monitoramento de modelos de Machine Learning.

Tópicos principais:

- scikit-learn.
- Pipelines.
- Cross-validation.
- MLflow.
- PyTorch.
- Custom datasets.
- DataLoaders.

- Training loop.
- DDP.
- Mixed precision.
- Quantization.
- ONNX export.
- Model serving.
- Monitoring.
- Drift.
- A/B testing.

Para que serve

Serve para transformar dados em modelos preditivos e disponibilizar esses modelos em sistemas reais.

Exemplos:

- Classificação.
- Regressão.
- Recomendação.
- Detecção de fraude.
- Previsão de demanda.
- Processamento de imagem.
- NLP.
- Ranking.
- Scoring.

Por que é importante

Criar um modelo em notebook é apenas uma parte pequena do trabalho. Em produção, é necessário:

- Versionar dados.
- Versionar modelos.
- Avaliar métricas.
- Servir o modelo.
- Monitorar drift.
- Reprocessar treinamento.
- Controlar experimentos.
- Integrar com sistemas reais.

Onde é usado em projetos reais**Exemplos:**

- Serviço de recomendação.
- Modelo de churn.
- Modelo de risco de crédito.
- Classificador de documentos.
- OCR + classificação.
- Sistema antifraude.
- Scoring de lead.
- Previsão financeira.
- Model serving via API.

Quais problemas resolve**Resolve:**

- Automação de decisões.
- Predição com base em dados.

- Classificação em escala.
- Redução de análise manual.
- Detecção de padrões.
- Personalização de experiência.

Erros comuns

Evite:

- Treinar modelo sem baseline.
- Não separar treino, validação e teste.
- Vazar dados entre treino e teste.
- Não versionar experimento.
- Não monitorar drift.
- Não medir performance em produção.
- Não controlar latência de inferência.
- Não pensar em fallback.
- Não avaliar custo de GPU/CPU.
- Não explicar limitações do modelo.

Conhecimentos prévios necessários

Python.

Estatística básica.

Álgebra linear básica.

Pandas/NumPy.

SQL.

Métricas de ML.

APIs.

Testes.

Docker/cloud.

Profundidade esperada para nível sênior

Um sênior precisa:

- Definir estratégia de validação.
- Comparar modelos com métricas adequadas.
- Projetar pipeline de treinamento.
- Usar MLflow para rastreabilidade.
- Servir modelos com segurança e observabilidade.
- Avaliar drift.
- Planejar rollback.
- Decidir entre batch inference, online inference e streaming inference.
- Usar ONNX, quantization ou mixed precision quando fizer sentido.
- Avaliar impacto de custo, latência e confiabilidade.

2.8 LLMs e RAG com Python

O que é

LLMs são modelos de linguagem capazes de gerar, interpretar, resumir e estruturar texto. RAG, ou Retrieval-Augmented Generation, é uma arquitetura que combina busca em base de conhecimento com geração de resposta por LLM.

Tópicos principais:

- LangChain.
- LangGraph.
- LlamaIndex.
- Vector stores.
- Embeddings.
- Tool calling.
- Agents.
- RAGAS.
- DeepEval.
- LangSmith.
- Langfuse.
- Guardrails.
- Structured output.
- Multimodal RAG.

Para que serve

Serve para criar sistemas inteligentes capazes de responder com base em dados privados, documentos internos, bases técnicas ou informações corporativas.

Aplicações:

- Chatbot com documentos.
- Assistente jurídico.
- Assistente técnico.
- Busca semântica.
- Extração estruturada.
- Geração de relatórios.
- Agentes com ferramentas.
- Atendimento automatizado.
- Análise de documentos.

Por que é importante

LLMs isolados podem gerar respostas incorretas ou desatualizadas. RAG reduz esse problema ao conectar o modelo com uma base de conhecimento controlada.

A arquitetura RAG bem construída exige:

- Ingestão de documentos.
- Chunking.
- Embeddings.
- Vector database.
- Retrieval.
- Reranking.
- Prompting.
- Geração.
- Avaliação.
- Observabilidade.
- Segurança.

Onde é usado em projetos reais**Exemplos:**

- API de perguntas e respostas sobre documentos.

- Assistente interno para suporte técnico.
- Chat com contratos.
- Busca em base de conhecimento.
- Copiloto de atendimento.
- Análise de PDFs.
- Extração de informações fiscais.
- Classificação de tickets.
- Agente que consulta APIs internas.

Quais problemas resolve

Resolve:

- Busca ineficiente em documentos.
- Dificuldade de acessar conhecimento interno.
- Respostas manuais repetitivas.
- Baixa produtividade em análise de texto.
- Falta de contextualização em LLM puro.
- Necessidade de automação com linguagem natural.

Erros comuns

Evite:

- Achar que LangChain resolve arquitetura sozinho.
- Usar RAG sem avaliar qualidade.
- Fazer chunking ruim.
- Não tratar permissões por documento.
- Não registrar fontes.
- Não proteger dados sensíveis.
- Não avaliar hallucination.
- Não aplicar guardrails.
- Não medir precisão de retrieval.
- Criar agentes autônomos sem controle.
- Não ter fallback quando o modelo falha.

Conhecimentos prévios necessários

Python.

APIs.

Banco de dados.

Vetores e similaridade.

HTTP.

Segurança.

Testes.

Observabilidade.

Noções de NLP.

Profundidade esperada para nível sênior

Um sênior precisa:

- Projetar arquitetura RAG fim a fim.
- Escolher vector store adequado.
- Definir estratégia de chunking.

- Avaliar embeddings.
- Implementar reranking quando necessário.
- Aplicar controle de acesso por documento.
- Medir qualidade com RAGAS ou DeepEval.
- Monitorar prompts e respostas com LangSmith/Langfuse.
- Definir guardrails.
- Controlar custo por token.
- Definir fallback e human-in-the-loop.
- Avaliar riscos de segurança, privacidade e compliance.

2.9 Testes, segurança e observabilidade

O que é

Essa área garante que aplicações Python sejam confiáveis, seguras, auditáveis e operáveis em produção.

Tópicos principais:

- pytest.
- fixtures.
- factory_boy.
- Hypothesis.
- Mutation testing.
- Snapshot tests.
- Bandit.
- Safety.
- pip-audit.
- SQL injection.
- SSRF.
- Deserialization attacks.
- OpenTelemetry.
- Sentry.
- structlog.

Para que serve

Serve para reduzir falhas, detectar vulnerabilidades, melhorar diagnóstico e aumentar confiança no deploy.

Inclui:

- Testes automatizados.
- Testes unitários.
- Testes de integração.
- Testes baseados em propriedades.
- Análise de dependências.
- Logs estruturados.
- Métricas.
- Tracing.
- Alertas.
- Monitoramento de erros.

Por que é importante

Código que funciona localmente pode falhar em produção por motivos como:

- Dados inesperados.
- Concorrência.

- Timeout.
- Dependência externa indisponível.
- Ataque de segurança.
- Configuração incorreta.
- Falta de logs.
- Falta de métricas.

O nível sênior exige pensar em operação desde o início.

Onde é usado em projetos reais

Exemplos:

- API FastAPI com testes de contrato.
- Django com testes de permissões.
- Pipeline com validação de dados.
- Serviço RAG com logs de prompt e resposta.
- Worker Celery com retry monitorado.
- API de IA com tracing distribuído.
- Deploy com alertas de erro.
- Verificação de CVEs no CI/CD.

Quais problemas resolve

Resolve:

- Bugs recorrentes.
- Falta de confiança no deploy.
- Vulnerabilidades em dependências.
- Falhas silenciosas.
- Dificuldade de debugar produção.
- Falta de rastreabilidade.
- Regressões.
- Problemas de segurança.

Erros comuns

Evite:

- Testar apenas caminho feliz.
- Não testar erro de autenticação.
- Não testar permissões.
- Não mockar dependências externas corretamente.
- Não validar payloads maliciosos.
- Não escanear dependências.
- Não sanitizar entrada.
- Logar dados sensíveis.
- Não usar correlation ID.
- Não configurar alertas.

Conhecimentos prévios necessários

Python.

pytest.

HTTP.

Segurança web.

Banco de dados.

Docker.

CI/CD.

Logs.

Métricas.

Profundidade esperada para nível sênior

Um sênior precisa:

- Definir estratégia de testes por camada.
- Revisar cobertura útil, não apenas percentual.
- Implementar testes de contrato.
- Usar property-based tests quando fizer sentido.
- Analisar vulnerabilidades.
- Implantar OpenTelemetry.
- Definir padrão de logs estruturados.
- Monitorar p95/p99, taxa de erro e saturação.
- Garantir segurança desde o design.
- Criar cultura de qualidade no time.

3. Organização como plano de estudos

Módulo 1 - Fundamentos sólidos de Python

Objetivo do módulo

Construir base forte em Python moderno, com foco em legibilidade, tipagem, testes e organização de projetos.

Conteúdos a estudar

- Sintaxe avançada.
- Funções.
- OOP.
- Magic methods.
- Iterators.
- Generators.
- Decorators.
- Context managers.
- dataclasses.
- typing.
- TypedDict.
- Literal.
- Protocol.
- Generic.
- Pydantic.
- Ambientes virtuais.
- Poetry.
- uv.
- Ruff.
- mypy.

- pyright.
- pytest.

Ordem recomendada

- Revisão de Python básico.
- OOP e composição.
- Funções avançadas.
- Iterators e generators.
- Decorators.
- Context managers.
- Tipagem.
- Pydantic.
- Estrutura de projeto.
- Testes com pytest.
- Lint, format e type checking.

Conceitos obrigatórios

- Mutabilidade.
- Escopo.
- Duck typing.
- Type hints.
- Protocolos.
- Injeção de dependência manual.
- Separação de responsabilidades.
- Testabilidade.

Práticas recomendadas

- Usar type hints em todo código novo.
- Rodar ruff, mypy ou pyright no CI.
- Separar domínio de infraestrutura.
- Escrever testes desde o início.
- Evitar funções genéricas demais.
- Nomear bem funções, classes e módulos.

Exercícios técnicos

- Criar uma lib de validação de CPF/CNPJ.
- Criar um parser de arquivos CSV.
- Criar decorators para logging e retry.
- Criar context manager para conexão simulada.
- Criar classes com Protocol.
- Criar testes parametrizados com pytest.

Mini projeto

SDK Python para integração com API externa

Funcionalidades:

- Cliente HTTP.
- Tipagem forte.
- Tratamento de erro.
- Retry.

- Logs.
- Testes.
- Empacotamento com Poetry ou uv.

Critérios de conclusão

Você conclui este módulo quando consegue:

- Criar projeto Python organizado.
- Aplicar tipagem com segurança.
- Escrever testes.
- Usar lint/type checker.
- Criar código modular e reutilizável.

Nível esperado ao finalizar

Intermediário forte em Python Core, pronto para APIs backend.

Módulo 2 - Aplicação prática com FastAPI e Django/DRF

Objetivo do módulo

Construir APIs robustas, seguras e testáveis usando FastAPI e Django/DRF.

Conteúdos a estudar

FastAPI:

- Routing.
- Pydantic v2.
- Dependency Injection.
- Middleware.
- Exception handlers.
- JWT.
- OAuth2.
- Uploads.
- Background tasks.
- TestClient.
- SQLAlchemy.
- Alembic.

Django/DRF:

- Models.
- Migrations.
- QuerySets.
- Serializers.
- ViewSets.
- Permissions.
- Filtering.
- Pagination.
- Transactions.
- Admin.
- Caching.

Ordem recomendada

- HTTP e REST.

- FastAPI básico.
- Pydantic v2.
- SQLAlchemy.
- Alembic.
- Autenticação JWT.
- Testes de API.
- Django Models.
- DRF Serializers/ViewSets.
- Permissions.
- Otimização de QuerySets.

Conceitos obrigatórios

- REST.
- Status codes.
- DTOs/schemas.
- Validação.
- Autenticação.
- Autorização.
- ORM.
- Migrations.
- Transações.
- Paginação.
- N+1 queries.

Práticas recomendadas

- Separar router, service, repository e schema.
- Não colocar regra de negócio no controller.
- Versionar APIs.
- Escrever testes de contrato.
- Usar migrations em qualquer alteração de banco.
- Validar permissões por endpoint.
- Implementar tratamento global de exceções.

Exercícios técnicos

- Criar CRUD com autenticação.
- Criar endpoint paginado e filtrado.
- Criar upload de arquivo.
- Criar middleware de correlation ID.
- Criar testes para erro 400, 401, 403 e 404.
- Otimizar QuerySet com select_related.

Mini projeto

API de gestão de documentos com autenticação

Funcionalidades:

- Cadastro de usuário.
- Login JWT.
- Upload de documento.
- Listagem paginada.

- Permissão por usuário.
- Logs estruturados.
- Testes automatizados.
- Banco com migrations.

Critérios de conclusão

Você conclui quando consegue:

- Criar API completa com autenticação.
- Persistir dados.
- Testar endpoints.
- Tratar exceções.
- Documentar via OpenAPI.
- Evitar N+1 no ORM.

Nível esperado ao finalizar

Backend intermediário/avançado com Python.

Módulo 3 - Aprofundamento em async, performance e dados

Objetivo do módulo

Dominar concorrência, otimização e manipulação de dados com Python.

Conteúdos a estudar

Async:

- Event loop.
- Coroutines.
- Tasks.
- Futures.
- Timeouts.
- Cancellation.
- Backpressure.
- httpx async.
- aiohttp.
- async SQLAlchemy.

Performance:

- cProfile.
- Py-Spy.
- Scalene.
- Memory profiling.
- GIL.
- Threading.
- Multiprocessing.
- NumPy.
- Polars.
- Numba.
- Cython.

Dados:

- Pandas.

- Polars.
- SQL.
- EDA.
- Feature engineering.
- Great Expectations.
- Batch processing.
- Streaming básico.

Ordem recomendada

- Diferença entre I/O-bound e CPU-bound.
- Event loop.
- Async com HTTP.
- Timeouts/retries.
- Profiling.
- GIL.
- Multiprocessing.
- Pandas.
- Polars.
- Validação de dados.
- Pipeline batch.

Conceitos obrigatórios

- Concorrência.
- Paralelismo.
- GIL.
- Latência.
- Throughput.
- Memory footprint.
- Data quality.
- Idempotência.
- Reprocessamento.

Práticas recomendadas

- Medir antes de otimizar.
- Usar async apenas para I/O.
- Usar multiprocessing para CPU-bound.
- Processar grandes arquivos em chunks.
- Validar schema de dados.
- Registrar métricas de pipeline.
- Controlar falhas e reprocessamento.

Exercícios técnicos

- Criar crawler assíncrono com limite de concorrência.
- Comparar Pandas e Polars.
- Medir performance com Py-Spy.
- Criar pipeline que valida CSV.
- Implementar retry com timeout.
- Criar processamento paralelo.

Mini projeto

Pipeline de ingestão de dados financeiros

Funcionalidades:

- Upload ou leitura de CSV/OFX.
- Validação de schema.
- Normalização.
- Deduplicação.
- Carga em banco.
- Logs.
- Métricas.
- Relatório de erros.
- Testes.

Critérios de conclusão

Você conclui quando consegue:

- Explicar quando usar async.
- Identificar gargalos.
- Otimizar processamento.
- Criar pipeline confiável.
- Validar dados.
- Reprocessar falhas com segurança.

Nível esperado ao finalizar

Avançado em Python aplicado a backend e dados.

Módulo 4 - Arquitetura, boas práticas e integração

Objetivo do módulo

Projetar sistemas Python sustentáveis, integráveis e evolutivos.

Conteúdos a estudar

- Clean Architecture.
- Arquitetura Hexagonal.
- DDD aplicado.
- Service layer.
- Repository pattern.
- Unit of Work.
- DTOs.
- Eventos.
- Mensageria.
- Integração com sistemas Java/Spring.
- Contratos OpenAPI.
- Versionamento de APIs.
- Modular monolith.
- Microsserviços.
- Cache.
- Transações.
- Idempotência.

Ordem recomendada

- Separação de camadas.
- Entidades e casos de uso.
- Ports/adapters.
- Repositories.
- Unit of Work.
- APIs e contratos.
- Eventos e mensageria.
- Integração com Java/Spring.
- Cache e consistência.
- Estratégias de evolução.

Conceitos obrigatórios

- Acoplamento.
- Coesão.
- Contratos.
- Idempotência.
- Consistência eventual.
- Transações distribuídas.
- Observabilidade arquitetural.
- Evolução incremental.

Práticas recomendadas

- Não acoplar domínio ao framework.
- Manter contratos versionados.
- Usar eventos para integrações assíncronas.
- Projetar idempotência em consumers.
- Aplicar testes por camada.
- Documentar decisões arquiteturais.
- Usar ADRs.

Exercícios técnicos

- Refatorar API FastAPI para arquitetura hexagonal.
- Criar adapter para banco.
- Criar adapter para LLM.
- Criar adapter para fila.
- Criar contrato OpenAPI.
- Criar consumer idempotente.
- Criar testes de caso de uso sem framework.

Mini projeto

Microserviço Python integrado a ecossistema Java

Funcionalidades:

- API FastAPI.
- Banco PostgreSQL.
- Mensageria Kafka ou RabbitMQ.
- Contratos OpenAPI.
- Autenticação JWT.

- Logs com correlation ID.
- Testes unitários e integração.
- Adapter para serviço externo Java/Spring.

Critérios de conclusão

Você conclui quando consegue:

- Projetar boundaries.
- Separar regra de negócio do framework.
- Criar integração robusta.
- Lidar com falhas externas.
- Justificar decisões arquiteturais.
- Criar arquitetura testável.

Nível esperado ao finalizar

Avançado com visão arquitetural.

Módulo 5 - Produção, performance, segurança e escalabilidade

Objetivo do módulo

Preparar aplicações Python para produção com segurança, observabilidade e capacidade de escala.

Conteúdos a estudar

- Docker.
- CI/CD.
- Health checks.
- Readiness/liveness.
- OpenTelemetry.
- Sentry.
- structlog.
- Métricas.
- Tracing.
- Rate limiting.
- Segurança em APIs.
- Bandit.
- Safety.
- pip-audit.
- SQL injection.
- SSRF.
- Deserialization attacks.
- Secrets management.
- Deploy cloud.
- Cache.
- Load testing.
- p95/p99 latency.

Ordem recomendada

- Docker.
- Configuração por ambiente.
- CI/CD.
- Logs estruturados.

- Métricas.
- Tracing.
- Segurança.
- Rate limiting.
- Performance em produção.
- Deploy cloud.
- Alertas.
- Runbooks.

Conceitos obrigatórios

- SLA.
- SLI.
- SLO.
- Error budget.
- Vulnerabilidades web.
- Gestão de secrets.
- Escalabilidade horizontal.
- Autoscaling.
- Observabilidade.
- Resiliência.

Práticas recomendadas

- Não logar dados sensíveis.
- Usar correlation ID.
- Escanear dependências no CI.
- Ter health checks.
- Monitorar latência p95/p99.
- Definir alertas úteis.
- Usar feature flags quando necessário.
- Ter rollback.
- Testar carga antes de escalar.

Exercícios técnicos

- Dockerizar API FastAPI.
- Criar pipeline CI.
- Adicionar OpenTelemetry.
- Integrar Sentry.
- Implementar rate limiting.
- Criar teste de carga.
- Rodar Bandit e pip-audit.
- Criar dashboard de métricas.

Mini projeto

API de IA pronta para produção

Funcionalidades:

- Upload de documento.
- Geração de embeddings.
- Busca semântica.

- Resposta via LLM.
- Autenticação.
- Rate limiting.
- Logs estruturados.
- Métricas.
- Tracing.
- Testes.
- Docker.
- CI/CD.
- Deploy cloud.

Critérios de conclusão

Você conclui quando consegue:

- Operar a aplicação em produção.
- Diagnosticar falhas.
- Monitorar performance.
- Proteger endpoints.
- Reduzir riscos.
- Planejar rollback.
- Escalar com critério.

Nível esperado ao finalizar

Avançado/sênior em entrega produtiva.

Módulo 6 - Domínio sênior, IA aplicada e liderança técnica

Objetivo do módulo

Consolidar domínio técnico sênior em IA aplicada, MLOps, RAG, decisões arquiteturais e liderança técnica.

Conteúdos a estudar

ML/MLOps:

- scikit-learn.
- PyTorch.
- Training loop.
- MLflow.
- Model serving.
- Drift.
- A/B testing.
- ONNX.
- Quantization.
- Monitoring.

LLM/RAG:

- LangChain.
- LangGraph.
- LlamaIndex.
- Embeddings.
- Vector stores.
- Tool calling.

- Agents.
- Guardrails.
- Structured output.
- RAGAS.
- DeepEval.
- LangSmith.
- Langfuse.
- Multimodal RAG.

Liderança técnica:

- ADRs.
- Code review.
- Mentoria.
- Padrões de arquitetura.
- Gestão de dívida técnica.
- Roadmap técnico.
- Decisão build vs buy.
- Avaliação de riscos.

Ordem recomendada

- scikit-learn pipelines.
- MLflow.
- Model serving.
- Monitoramento de drift.
- PyTorch básico/intermediário.
- ONNX e otimização de inferência.
- RAG básico.
- RAG avançado.
- Avaliação de RAG.
- Guardrails.
- Agentes.
- Arquitetura de produto com IA.
- Liderança técnica.

Conceitos obrigatórios

- Dataset versioning.
- Model registry.
- Experiment tracking.
- Drift.
- Retrieval.
- Reranking.
- Prompt evaluation.
- Guardrails.
- Token cost.
- Latência de LLM.
- Segurança de dados.
- Human-in-the-loop.
- Governança técnica.

Práticas recomendadas

- Não colocar modelo em produção sem monitoramento.
- Não usar LLM sem avaliação.
- Registrar fontes em RAG.
- Aplicar controle de acesso por documento.
- Controlar custo por token.
- Criar fallback.
- Documentar decisões.
- Revisar arquitetura com o time.
- Planejar evolução incremental.

Exercícios técnicos

- Criar modelo scikit-learn com MLflow.
- Servir modelo via FastAPI.
- Monitorar drift.
- Criar RAG com vector store.
- Avaliar RAG com RAGAS.
- Criar tool calling controlado.
- Criar structured output com validação.
- Criar ADR para escolha de arquitetura.

Mini projeto

Plataforma de IA corporativa com RAG e MLOps

Funcionalidades:

- Upload de documentos.
- Pipeline de ingestão.
- Chunking.
- Embeddings.
- Vector store.
- Busca semântica.
- Reranking.
- Resposta via LLM.
- Guardrails.
- Avaliação com RAGAS/DeepEval.
- Logs com Langfuse/LangSmith.
- API FastAPI.
- Autenticação.
- Observabilidade.
- Deploy.
- Documentação arquitetural.

Critérios de conclusão

Você conclui quando consegue:

- Projetar um produto de IA com arquitetura real.
- Avaliar qualidade de RAG.
- Controlar custo, segurança e latência.
- Integrar IA com backend.
- Definir estratégia de MLOps.

- Liderar decisões técnicas.
- Revisar arquitetura de outros desenvolvedores.

Nível esperado ao finalizar

Sênior em Python Backend + IA aplicada.

4. Critérios de domínio técnico por grande área

Python Core

Nível	Critério
Iniciante	Entende sintaxe, funções, classes e estruturas básicas
Intermediário	Escreve módulos organizados, usa ambientes virtuais, pytest e type hints
Avançado	Usa Protocol, Generic, decorators, context managers e packaging
Sênior	Define padrões de código, arquitetura modular, tipagem, testes e qualidade no CI

Backend Python

Nível	Critério
Iniciante	Cria endpoints simples
Intermediário	Cria CRUD com banco, validação e autenticação
Avançado	Cria APIs testáveis, seguras, versionadas e observáveis
Sênior	Projeta arquitetura, contratos, integração, escalabilidade e governança técnica

Async e performance

Nível	Critério
Iniciante	Entende async/await conceitualmente
Intermediário	Usa httpx/aiohttp async corretamente
Avançado	Controla timeouts, cancellation, concorrência e profiling
Sênior	Decide entre async, threads, processos, filas e otimização nativa com base em trade-offs

Dados e pipelines

Nível	Critério
Iniciante	Manipula CSV e DataFrames
Intermediário	Cria transformações e consultas SQL
Avançado	Cria pipelines validados, idempotentes e monitorados
Sênior	Projeta arquitetura de dados, qualidade, lineage, reprocessamento e custo operacional

ML/MLOps

Nível	Critério
Iniciante	Treina modelos simples
Intermediário	Usa pipelines, validação cruzada e métricas
Avançado	Versiona experimentos, serve modelos e monitora performance
Sênior	Define estratégia de MLOps, drift, A/B testing, rollback, custo e governança

LLM/RAG

Nível	Critério
Iniciante	Consome API de LLM
Intermediário	Cria RAG básico com embeddings e vector store
Avançado	Avalia retrieval, usa guardrails e structured output
Sênior	Projeta RAG seguro, escalável, avaliado, observável e economicamente viável

Segurança e observabilidade

Nível	Critério
Iniciante	Entende logs e erros comuns
Intermediário	Escreve testes e valida entradas
Avançado	Usa scanners, tracing, métricas e logs estruturados
Sênior	Define estratégia de segurança, SLOs, alertas, resposta a incidentes e padrões de produção

5. Aplicação prática em projetos reais

APIs

Aplicação:

- FastAPI para APIs de IA.
- Django/DRF para sistemas administrativos.
- OpenAPI para documentação.
- JWT/OAuth2 para autenticação.
- Rate limiting para proteção.

Exemplo real:

- API para upload de documento, extração de texto, geração de embeddings, busca semântica e resposta via LLM.

Banco de dados

Aplicação:

- PostgreSQL para dados relacionais.
- SQLAlchemy/Alembic para FastAPI.
- Django ORM para sistemas Django.

- Índices para performance.
- Transações para consistência.

Exemplo real:

- Sistema financeiro com transações, categorias, importações e relatórios.

Microsserviços

Aplicação:

- Serviço Python especializado em IA.
- Serviço Java/Spring como core transacional.
- Comunicação via REST ou mensageria.
- Contratos versionados.
- Observabilidade distribuída.

Exemplo real:

- Java gerencia pedidos e Python executa classificação inteligente de documentos fiscais.

Mensageria**Aplicação:**

- Kafka/RabbitMQ para processamento assíncrono.
- Consumers idempotentes.
- Retry.
- Dead-letter queue.
- Backpressure.

Exemplo real:

- Evento DocumentUploaded dispara pipeline Python de OCR, embeddings e indexação.

Segurança

Aplicação:

- JWT.
- OAuth2.
- Validação de entrada.
- Proteção contra SQL injection.
- Proteção contra SSRF.
- Sanitização de upload.
- Controle de acesso por documento.

Exemplo real:

- RAG corporativo onde cada usuário só pode consultar documentos autorizados.

Observabilidade**Aplicação:**

- OpenTelemetry.
- structlog.
- Sentry.
- Métricas de latência.
- Tracing distribuído.
- Correlation ID.
- Logs de prompts e respostas.

Exemplo real:

- Monitorar por que uma resposta de LLM ficou lenta ou incorreta rastreando upload, chunking, retrieval e geração.

Testes

Aplicação:

- Unitários para regra de negócio.
- Integração para banco.
- Contrato para APIs.
- Property-based tests.
- Testes de segurança.
- Testes de RAG.

Exemplo real:

- Validar se um pipeline financeiro nunca gera transação duplicada mesmo com reprocessamento.

Deploy

Aplicação:

- Docker.
- CI/CD.
- Variáveis por ambiente.
- Health checks.
- Autoscaling.
- Rollback.

Exemplo real:

- Deploy de API FastAPI com worker separado para processamento pesado.

Arquitetura

Aplicação:

- Hexagonal Architecture.
- Clean Architecture.
- Modular monolith.
- Microserviços.
- Event-driven architecture.
- MLOps architecture.

Exemplo real:

- Serviço Python de IA isolado do core transacional Java, comunicado por eventos e contratos HTTP.

Performance

Aplicação:

- Profiling.
- Cache.
- Batch processing.
- Polars.
- ONNX Runtime.
- Async I/O.
- Multiprocessing.

Exemplo real:

- Reduzir tempo de processamento de arquivos financeiros de 30 minutos para 3 minutos usando Polars, chunking e paralelismo controlado.

Manutenção evolutiva

Aplicação:

- Versionamento de APIs.
- Testes de regressão.
- ADRs.
- Refatoração segura.
- Observabilidade.
- Feature flags.

Exemplo real:

- Evoluir um RAG simples para RAG com reranking e avaliação sem quebrar o contrato da API.

6. Profundidade sênior: trade-offs e decisões

FastAPI vs Django/DRF

Escolha	Quando usar
FastAPI	APIs modernas, serviços de IA, async, microsserviços
Django/DRF	Sistemas administrativos, CRUD robusto, backoffice, permissões complexas

Decisão sênior:

- FastAPI entrega mais flexibilidade e performance em APIs especializadas.
- Django entrega mais produtividade em aplicações completas.
- O erro é usar FastAPI para reconstruir um Django inteiro ou usar Django para um serviço pequeno de IA que precisa de baixa latência e async.

Async vs fila

Escolha	Quando usar
Async	Muitas chamadas I/O dentro da mesma requisição
Fila	Tarefas demoradas, reprocessáveis ou críticas
Multiprocessing	CPU-bound
Threading	I/O legado ou bibliotecas bloqueantes

Decisão sênior:

- Não use async como substituto de arquitetura assíncrona.
- Para processamento demorado, use fila.
- Para chamada externa rápida, async pode ser suficiente.
- Para IA pesada, use worker separado.

Pandas vs Polars

Escolha	Quando usar
Pandas	Ecossistema maduro, análises comuns, compatibilidade
Polars	Grandes volumes, performance, lazy execution

Decisão sênior:

- Pandas é excelente para prototipação.
- Polars pode ser melhor para pipelines pesados.
- A decisão deve considerar volume, memória, time, compatibilidade e custo de manutenção.

RAG simples vs RAG avançado

Escolha	Quando usar
RAG simples	Base pequena, baixo risco, protótipo
RAG com reranking	Base grande, precisão importante
RAG com avaliação	Produto real
RAG com controle de acesso	Ambiente corporativo

Decisão sênior:

- RAG sem avaliação não é confiável.
- RAG sem controle de permissão pode vazar informação.
- RAG sem observabilidade é difícil de evoluir.
- O custo por token precisa ser monitorado.

Modelo próprio vs API externa

Escolha	Vantagem	Risco
API externa	Velocidade, qualidade, menor operação	Custo, privacidade, dependência
Modelo próprio	Controle, privacidade, customização	Infra, MLOps, custo operacional

Decisão sênior:

- Para começar, API externa costuma acelerar validação.
- Para dados sensíveis, compliance e escala, modelo próprio pode fazer sentido.
- A decisão deve considerar custo, latência, privacidade, qualidade e time técnico.

7. Cronograma sugerido

Plano intensivo de 24 semanas

Semanas 1-4 - Python Core avançado

Prioridade: máxima.

Estudar:

- Tipagem.
- OOP.
- Decorators.
- Generators.
- Context managers.
- Pydantic.
- Poetry/uv.
- Ruff.
- mypy/pyright.
- pytest.

Checkpoint:

- Criar SDK Python testado e tipado.

Semanas 5-8 - FastAPI

Prioridade: máxima.

Estudar:

- Routing.
- Schemas.
- Dependency Injection.
- SQLAlchemy.
- Alembic.
- JWT.
- Uploads.
- Middleware.
- TestClient.
- Observabilidade inicial.

Checkpoint:

- Criar API de documentos com autenticação e banco.

Semanas 9-11 - Django/DRF

Prioridade: média/alta.

Estudar:

- Models.
- Migrations.
- Serializers.
- ViewSets.
- Permissions.
- QuerySets.
- Transactions.
- Caching.
- N+1.

Checkpoint:

- Criar backoffice/API administrativa com DRF.

Semanas 12-14 - Async e performance

Prioridade: alta.

Estudar:

- Event loop.
- Tasks.
- Timeouts.
- Cancellation.
- Backpressure.
- Profiling.
- GIL.
- Multiprocessing.
- Polars.
- NumPy.

Checkpoint:

- Criar crawler/pipeline assíncrono com métricas de performance.

Semanas 15-17 - Dados e pipelines

Prioridade: alta.

Estudar:

- Pandas.
- Polars.
- SQL.
- EDA.
- Feature engineering.
- Great Expectations.
- Airflow/Prefect/Dagster.
- dbt.
- Batch.
- Streaming básico.

Checkpoint:

- Criar pipeline financeiro com validação e carga.

Semanas 18-20 - ML e MLOps

Prioridade: alta para IA aplicada.

Estudar:

- scikit-learn.
- Pipelines.
- Cross-validation.
- MLflow.
- PyTorch.
- Model serving.
- Drift.
- A/B testing.
- ONNX.

Checkpoint:

- Criar modelo versionado, servido via FastAPI e monitorado.

Semanas 21-23 - LLMs e RAG

Prioridade: máxima para IA moderna.

Estudar:

- Embeddings.
- Vector stores.
- LangChain.
- LangGraph.
- LlamaIndex.
- RAGAS.
- DeepEval.
- Guardrails.
- Structured output.
- LangSmith/Langfuse.

Checkpoint:

- Criar API RAG com avaliação, autenticação e logs.

Semana 24 - Projeto final e revisão sênior

Prioridade: consolidação.

Entregar:

- Projeto completo.
- Documentação técnica.
- ADRs.
- Diagramas.
- Testes.
- Observabilidade.
- Deploy.
- Checklist de segurança.
- Análise de trade-offs.

Checkpoint final:

- Apresentar o projeto como se fosse para uma banca técnica ou entrevista sênior.

8. Projetos recomendados para portfólio

Projeto 1 - API Python profissional com FastAPI

Recursos:

- FastAPI.
- PostgreSQL.
- SQLAlchemy.
- Alembic.
- JWT.
- Upload.
- Testes.
- Docker.
- OpenAPI.
- Logs estruturados.

Demonstra:

- Backend Python sólido.
- Arquitetura limpa.
- Testabilidade.
- Produção básica.

Projeto 2 - Pipeline financeiro com validação

Recursos:

- Pandas/Polars.
- SQL.
- Great Expectations.
- Airflow/Prefect.
- PostgreSQL.
- Logs.

- Reprocessamento.
- Idempotência.

Demonstra:

- Engenharia de dados.
- Data quality.
- Processamento batch.
- Confiabilidade.

Projeto 3 - Serviço de ML com MLOps

Recursos:

- scikit-learn.
- MLflow.
- FastAPI.
- Docker.
- Model registry.
- Monitoramento.
- Drift básico.
- Testes.

Demonstra:

- ML aplicado.
- Deploy de modelo.
- Operação de IA.

Projeto 4 - Plataforma RAG corporativa

Recursos:

- Upload de documentos.
- Chunking.
- Embeddings.
- Vector store.
- LLM.
- Reranking.
- RAGAS/DeepEval.
- Guardrails.
- Langfuse/LangSmith.
- Autenticação.
- Controle de acesso.
- Observabilidade.

Demonstra:

- IA aplicada moderna.
- Arquitetura de produto.
- Segurança.
- Avaliação de qualidade.

Projeto 5 - Microsserviço Python integrado com Java/Spring

Recursos:

- FastAPI.

- Kafka/RabbitMQ.
- REST contract.
- PostgreSQL.
- Docker.
- OpenTelemetry.
- Testes de integração.
- Idempotência.
- Retry/DLQ.

Demonstra:

- Integração enterprise.
- Microsserviços.
- Mensageria.
- Arquitetura distribuída.

9. Checklist de domínio técnico

Python Core

- I Escrevo código tipado.
- I Uso pytest com fixtures.
- I Uso ruff, mypy ou pyright.
- I Sei criar pacotes Python.
- I Entendo decorators, generators e context managers.
- I Sei aplicar Protocol, Generic, TypedDict e Pydantic.
- Backend I Crio APIs FastAPI com arquitetura limpa.
- I Crio APIs Django/DRF com permissões.
- I Uso migrations.
- I Testo endpoints.
- I Trato exceções.
- I Implemento autenticação.
- I Evito N+1.
- Async/performance I Sei quando usar async.
- I Configuro timeouts.
- I Evito bloquear event loop.
- I Faço profiling.
- I Entendo GIL.
- I Uso multiprocessing quando necessário.
- I Otimizo DataFrames com critério.

Dados

- I Crio pipelines idempotentes.
- I Valido dados.
- I Trato falhas.

- I Registro logs.
- I Sei reprocessar.
- I Entendo batch vs streaming.

ML/MLOps

- I Crio pipeline de ML.
- I Uso cross-validation.
- I Versiono experimentos.
- I Sirvo modelo via API.
- I Monitoro métricas.
- I Entendo drift.
- I Planejo rollback.

LLM/RAG

- I Crio embeddings.
- I Uso vector store.
- I Implemento RAG.
- I Avalio qualidade.
- I Uso guardrails.
- I Controlo custo por token.
- I Implemento controle de acesso.
- I Monitoro prompts e respostas.
- Segurança e produção I Uso Bandit/pip-audit.
- I Protejo contra SQL injection.
- I Protejo contra SSRF.
- I Não logo dados sensíveis.
- I Uso OpenTelemetry.
- I Tenho logs estruturados.
- I Tenho health checks.
- I Tenho CI/CD.
- I Tenho estratégia de rollback.

10. Indicadores de prontidão para nível

avançado/sênior

Você estará pronto para atuar em nível avançado/sênior quando conseguir:

- Projetar uma API Python do zero com autenticação, banco, testes, logs, métricas e deploy.
- Explicar trade-offs entre FastAPI, Django, fila, async, multiprocessing e microserviços.
- Criar pipelines de dados confiáveis com validação, idempotência e reprocessamento.
- Servir modelos de ML em produção com versionamento, monitoramento e fallback.
- Construir RAG com avaliação real, controle de acesso, guardrails e observabilidade.
- Integrar Python com sistemas Java/Spring via REST, eventos ou mensageria.

- Diagnosticar problemas de produção usando logs, métricas e tracing.
- Revisar arquitetura e código de outros devs com argumentos técnicos.
- Definir padrões técnicos para o time.
- Justificar decisões considerando custo, segurança, performance, manutenção e escalabilidade.

Resumo final da trilha Esta trilha forma um profissional capaz de atuar com Python Backend + IA em nível avançado/sênior, conectando engenharia de software, dados, machine learning, LLMs, arquitetura e produção.

A sequência ideal é:

- Python Core avançado.
- FastAPI.
- Django/DRF.
- Async e performance.
- Dados e pipelines.
- ML/MLOps.
- LLMs/RAG.
- Testes, segurança e observabilidade.
- Arquitetura e liderança técnica.

O resultado esperado é um profissional capaz de não apenas escrever código Python, mas projetar, entregar, operar e evoluir sistemas Python com IA em ambiente real de produção.